

Threads e Sincronização

Integrantes do grupo: Matheus Dias Pichler, Gabriel Argimon Gonzalez, Gabriel Segala e Artur Kielling

No universo dos Sistemas Operacionais, a eficiência é a palavra de ordem. Para aproveitar ao máximo os processadores modernos, não basta executar uma tarefa por vez; precisamos da **multiprogramação**. Através do uso de **threads**, conseguimos fazer com que um programa realize diversas atividades simultaneamente, como processar dados enquanto espera uma resposta de entrada ou saída.

Neste trabalho, exploramos como aplicar esses conceitos na prática, utilizando a biblioteca **Pthread** em linguagem **C** para resolver problemas matemáticos e entender os riscos da execução compartilhada.

1. O Perigo da Concorrência: A Seção Crítica

Nem tudo são flores no mundo multithread. Quando duas threads tentam mexer no mesmo dado ao mesmo tempo, as coisas podem sair do controle. É o que chamamos de **Seção Crítica**: um trecho de código que acessa uma variável compartilhada e que, se não for vigiado, gera resultados imprevisíveis.

Processos cont = 5

Imagine dois processos acessando uma contagem que começa em 5. O Processo A quer somar 1, e o Processo B quer subtrair 1.

- **a) Qual deveria ser o valor final?** Matematicamente, o resultado deveria ser **5**. Como uma operação anula a outra (+1 e -1), o sistema deveria retornar ao ponto de partida após as duas threads terminarem.
- **b) O "Atropelo" (Por que o valor fica errado?):** Na prática, sem sincronização, o resultado pode acabar sendo **4** ou **6**. Isso acontece porque as operações `cont++` e `cont--` não são "mágicas" ou instantâneas para o processador; elas ocorrem em três etapas: **Ler** o valor, **Modificar** e **Gravar** de volta.

Uma possível execução errada:

1. O **Processo A** lê o valor 5.
2. O **Processo B** também lê o valor 5, antes de o Processo A terminar sua tarefa.
3. O **Processo A** soma 1 e grava 6 na memória.

4. O **Processo B**, que ainda tem aquele 5 guardado na cabeça, subtrai 1 e grava 4.
5. **Resultado:** O valor 6 foi sobrescrito pelo 4, e o incremento do Processo A "sumiu" do sistema.

O Saldo Bancário

O mesmo risco aparece em operações financeiras, como no comando `saldo = saldo - valor;`.

- **É uma Seção Crítica?** Com certeza. Se dois saques de R\$ 10 acontecem num saldo de R\$ 100, o correto é sobrar R\$ 80.
- **O Erro na Prática:** Sem proteção, ambos os processos podem ler "100" ao mesmo tempo e ambos gravarem "90" no final. Um dos saques "some" do registro porque os processos não esperaram sua vez de atualizar o valor.

2. A Solução:

Para resolver esses impasses, utilizamos a Exclusão Mútua. Ele funciona como uma chave: a thread que chega primeiro "tranca" a variável, realiza sua tarefa com segurança e só depois "destranca" para a próxima. Nos nossos códigos `a_com_sinc.c` e `b_com_sinc.c`, mostramos como essa técnica garante que o resultado final seja sempre o esperado, mantendo a integridade dos dados.